Claude Agent SDK & Agentic Orchestration by Michaël Bettan

Definition: The **Claude Agent SDK** (TypeScript `@anthropic-ai/claude-agent-sdk` · Python `claude-agent-sdk`, which replaces `claude-code-sdk`) packages the Claude Code agent harness as a library: the agent loop, built-in tools, permissions, hooks, subagents, sessions and MCP, running inside **your** process. This guide covers the SDK API and the architecture patterns you build with it: loops, coordinator–subagent orchestration, deterministic enforcement, session state, tool error design, escalation, error propagation, long-context management and provenance.

Three ways to wire an agent:

Wiring path	Who runs the loop	You own	Pick when
Raw Messages API loop	Your code	Tool execution, `stop_reason` handling, state, retries, compaction	Full control, minimal dependencies
Claude Agent SDK	SDK in your process	Hosting, filesystem isolation, storage	You want Claude Code's tools, hooks, subagents and sessions in your own infrastructure
Claude Managed Agents	Anthropic server-side	Agent config, custom tool handlers, session lifecycle	Long-running hosted sandbox sessions without running containers yourself

MENTAL MODEL
The model *decides* (which tool, which subagent, when to stop); the harness *enforces* (permissions, hooks, budgets). Anything that must hold 100% of the time belongs in code (hooks, tool scoping, prerequisite gates), never only in the prompt.

SCOPE NOTE
Claude Code configuration (permission modes, `CLAUDE.md`, skills, plugins, CLI flags) lives in the Claude Code guide. MCP protocol internals live in the MCP guide.

1. The agentic loop and `stop_reason`

Raw Messages API loop (what the SDK does for you):

1. **Call:** send `messages` + `tools` to Claude.
2. **Inspect:** read `stop_reason` on the response.
3. **If `"tool_use"`:** execute **every** `tool_use` block, append the assistant turn, then append one user turn containing a `tool_result` block per call (matched by `tool_use_id`). Go to 1.
4. **If `"end_turn"`:** the model is done. Use the final text.

`stop_reason`	Meaning	Loop action
`tool_use`	Claude wants tools run	Execute, append `tool_result`s, call again
`end_turn`	Natural completion	Stop; return the answer
`max_tokens`	Hit your `max_tokens` cap mid-response	Raise the cap or continue the response; output is truncated
`pause_turn`	A **server-tool** loop hit its iteration limit	Send the assistant content back unchanged to let Claude continue
`refusal`	Claude declined	Handle explicitly (read `stop_details`, fallback model or human)
`stop_sequence`	One of your `stop_sequences` fired	Read `stop_sequence`
`model_context_window_exceeded`	Response filled the context window	Treat as truncated; compact or split the task

DECISION RULE
The loop's termination signal is `stop_reason`, **not** the text. Continue on `tool_use`; stop on `end_turn`; handle the rest explicitly.

EXAM TRAPS (LOOP ANTI-PATTERNS)
(1) parsing natural language ("I'm done", "task complete") to decide termination; (2) treating "the response contains text" as completion (Claude often writes text *and* calls tools in the same turn); (3) using an arbitrary iteration cap as the **primary** stop condition (keep a cap only as a safety bound); (4) dropping tool results instead of appending them, so the model cannot reason about what happened.

Model-driven vs pre-configured: In an agent loop Claude chooses the next tool from context. A fixed decision tree or hard-coded tool sequence is a *workflow*, not an agent; use it only when the order is known in advance (see §3).

The SDK version of the loop: `query()` runs the loop and streams messages (`AssistantMessage`, `ResultMessage`, ...). Check `Result Message.subtype` first:

`ResultMessage.subtype`	What happened	`result` present?
`success`	Finished normally	Yes
`error_max_turns`	Hit `max_turns` / `maxTurns`	No
`error_max_budget_usd`	Hit `max_budget_usd` / `maxBudgetUsd` (subagent spend counts toward the cap)	No
`error_during_execution`	Loop interrupted (cancelled request, crash)	No

`ResultMessage.subtype`	What happened	`result` present?
`error_max_structured_output_retries`	No schema-valid structured output within the retry limit	No

- Every result carries `session_id`, `total_cost_usd`, `usage`, `num_turns`, so you can resume even after an error. The result also has a `stop_reason` (`end_turn`, `max_tokens`, `refusal`, ...).
- A single-shot `query()` **raises after yielding an error result**: wrap the loop in `try` if you need to continue (for example, resume with a higher `max_turns`).

2. Agent SDK core API

- **`query(prompt, options)` vs `ClaudeSDKClient`:**
- `query()` starts a fresh session per call unless you pass **`continue_conversation=True`** (`continue: true` in TS), **`resume=session_id`** (from `ResultMessage.session_id`), or **`resume=session_id, fork_session=True`** (`forkSession: true`) to branch history into a new session ID. **Fork trap:** `fork_session` branches the conversation transcript **only**, not the filesystem. Pair with `enable_file_checkpointing=True` + `rewind_files(user_message_id)` to restore disk state.
- **`ClaudeSDKClient`** (Python `async with ClaudeSDKClient(options) as client:`): reuses one session across multiple `await client.query(...)` + `client.receive_response()` turns and supports **`await client.interrupt()`** (**drain buffered messages** up to the interrupted turn's `ResultMessage` before sending the next query). TypeScript has no `ClaudeSDKClient` class; call methods on the `Query` object returned by `query()`.

`setting_sources` / `settingSources` (`"user" | "project" | "local"`):

- **Omitted (default)** → loads `user`, `project` and `local` settings, `CLAUDE.md`, skills, agents and hooks, **just like the interactive CLI**. *(Exception: if top-level `skills` is passed while `setting_sources` is omitted, only `user` and `project` load.)*
- **`[]` (empty list)** → disables `user`, `project` and `local` filesystem settings and `CLAUDE.md`. **Multi-tenant trap — what `setting_sources=[]` still loads:** (1) **managed policy settings**, (2) **`~/.claude.json`** global state (isolate via `CLAUDE_CONFIG_DIR` in `env`), (3) **auto memory** `MEMORY.md` (disable with **`CLAUDE_CODE_DISABLE_AUTO_MEMORY="1"`** or `autoMemoryEnabled: false`), and (4) **`claude.ai` MCP connectors** (`mcp_servers={}` does *not* suppress them; pass **`strict_mcp_config=True`** / `strictMcpConfig: true`).

Option (Python / TS)	What it controls & critical SDK traps
`tools`	**Built-in tool availability in context** (`["Read", "Grep"]`, or `[]` to strip all built-ins so Claude sees only MCP tools).
`allowed_tools` / `allowedTools`	**Pre-approves** matching calls; unlisted tools stay in context and fall through to `permission_mode` / `can_use_tool`. No effect in `bypassPermissions`.
`disallowed_tools` / `disallowedTools`	Bare names (`"Bash"`) **remove the tool from context**; scoped rules (`"Bash(rm *)"`) keep the tool and **deny matching calls in every mode**, including `bypassPermissions`. File-write rules must use `Edit(//abs/path)` (`Write(...)` never matches).
`permission_mode` / `permissionMode`	`"default"`, `"acceptEdits"`, `"plan"`, `"auto"`, `"dontAsk"`, `"bypassPermissions"`. In TS, `"bypassPermissions"` throws unless **`allowDangerouslySkipPermissions: true`** is also set.
`can_use_tool` / `canUseTool`	Async callback returning `PermissionResultAllow(updated_input=...)` or `PermissionResultDeny(message=...)`. Fires **only** when the harness would prompt; **never** in `dontAsk`, `bypassPermissions`, or for tools matched by `allowed_tools`.
`hooks`	In-process callbacks (`PreToolUse`, `PostToolUse`, `Stop`, `SubagentStop`, `UserPromptSubmit`, ...) registered with `HookMatcher` (see §5).
`mcp_servers` + `create_sdk_mcp_server`	External or **in-process custom tools** (`@tool` + `create_sdk_mcp_server(name="srv", tools=[fn])`; TS `tool()` + `createSdkMcpServer()`). Pre-approve as **`mcp__<srv>__<tool>`**.
`agents` (`AgentDefinition`)	Programmatic subagents (see §4). **Python quirk:** multi-word fields use **camelCase** (`disallowedTools`, `mcpServers`, `maxTurns`, `permissionMode`, `initialPrompt`).
`max_turns` / `max_budget_usd`	Safety bounds; produce `error_max_turns` / `error_max_budget_usd`.
`output_format` / `outputFormat`	`{"type": "json_schema", "schema": ...}` → validated payload in `ResultMessage.structured_output`.
`env`	**Python merges** `env` onto the process environment; **TypeScript replaces** `process.env` — spread `{ ...process.env, MY_VAR: "1" }` or `PATH` is lost and the CLI fails with `ENOENT`.

3. Coordinator–subagent orchestration and task decomposition

Hub-and-spoke: one **coordinator** decomposes the task, decides which subagents to invoke, passes each one the context it needs, aggregates results and handles errors. Subagents never talk to each other directly; **all routing goes through the coordinator** (observability, consistent error handling, controlled information flow).

Coordinator responsibility	Do	Avoid
Select	Pick subagents based on what the query needs (simple question → one agent)	Always running the full pipeline for every request
Partition	Give each subagent a distinct subtopic or source type	Overlapping scopes → duplicated work and conflicting findings
Decompose	Cover the whole question; check the decomposition against the original goal	Overly narrow subtasks that silently drop parts of a broad topic
Instruct	State goals, scope and quality criteria	Step-by-step procedures that stop subagents from adapting
Refine	Evaluate the synthesis for gaps, re-delegate targeted queries, re-synthesize	Accepting the first synthesis even when coverage is thin

Choosing a decomposition pattern:

Pattern	Use when	Example
Prompt chaining (fixed sequential pipeline)	Steps and order are known in advance; predictable multi-aspect work	Review each file individually → separate cross-file integration pass
Dynamic / adaptive decomposition	Open-ended investigation; next steps depend on findings	"Add tests to a legacy codebase": map structure → find high-impact areas → prioritized plan that adapts as dependencies appear
Parallel fan-out	Subtasks are independent	Search, document analysis and data lookup at the same time

EXAM TRAP — ATTENTION DILUTION

One giant prompt reviewing 30 files at once gives shallow, inconsistent findings. Split into **per-file local passes** plus **one cross-file integration pass**.

4. Subagent invocation, `AgentDefinition` and context passing

```python
from claude_agent_sdk import query, ClaudeAgentOptions,
AgentDefinition

options = ClaudeAgentOptions(
    allowed_tools=["Read", "Grep", "Glob", "Agent"],  # the
coordinator needs the Agent tool
    agents={
        "code-reviewer": AgentDefinition(
            description="Security and quality review. Use for code
review tasks.",  # WHEN to delegate
            prompt="You are a code review specialist...",  #
subagent system prompt
            tools=["Read", "Grep", "Glob"],  # least privilege
            model="sonnet",
        ),
    },
)
```

- **Spawning mechanism:** Claude spawns subagents through the **`Agent` tool** (legacy name `Task`, still accepted as an alias; older docs and exams say "Task tool"). The coordinator can only delegate if that tool is **in context** (with an explicit `tools` list, include `"Agent"`; a bare `disallowed_tools` entry removes it) **and approved** (docs examples also list `"Agent"` in `allowed_tools`, which only pre-approves calls).
- **`AgentDefinition` fields:** `description` (required; tells the coordinator *when* to use it), `prompt` (required; system prompt), `tools` (omit = inherit all), `disallowedTools`, `model` (`"sonnet"`, `"opus"`, `"haiku"`, `"inherit"`, or full ID), `skills`, `mcpServers`, `maxTurns`, `memory`, `background`, `effort`, `permissionMode`, `omitClaudeMd` (TS).
- **Fan-out limits:** spawn depth and concurrency are capped by `CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH` and `CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS` (verify current defaults).

A (non-fork) subagent receives	It does not receive
Its own system prompt (`AgentDefinition.prompt`) + the **Agent tool's prompt string**	The parent's conversation history or tool results
Project `CLAUDE.md` (via `setting_sources`, unless `omitClaudeMd`)	The parent's system prompt
Its tool definitions (inherited or the `tools` subset)	Skill content, unless listed in `skills`

DECISION RULE — CONTEXT PASSING

The **only** channel from coordinator to subagent is the prompt string. Put everything the subagent needs **in that prompt**: complete prior findings, file paths, error messages, decisions. Use a structured format that separates **content** from **metadata** (source URL, document name, page, date) so attribution survives.

- **Return path:** only the subagent's **final message** returns to the coordinator; intermediate tool output is discarded. Ask subagents to return structured results (or write artifacts to disk and return the path).
- **Parallel subagents:** emit **multiple `Agent` tool calls in a single coordinator response** to run them concurrently. Spreading them across separate turns serializes the work.
- **Fork vs fresh subagent:** a *fork* inherits the parent conversation; a normal subagent starts fresh. Use fresh subagents for isolation and independent review.

5. Hooks: deterministic enforcement vs prompt guidance

Need	Mechanism	Why
A rule that must **always** hold (identity verified before refund, refund cap, no writes to `.env`)	**`PreToolUse` hook** (or tool scoping / prerequisite gate in code)	Prompts are probabilistic: a non-zero share of runs will skip or misread the instruction
Normalize or redact tool output before the model sees it	**`PostToolUse` hook** (`updatedToolOutput`, `additionalContext`)	One place to fix heterogeneous formats (Unix epoch vs ISO 8601, numeric status codes)
Style, tone, preferred approach, soft priorities	**System prompt / `CLAUDE.md`**	Guidance; occasional deviation is acceptable

```python
from claude_agent_sdk import ClaudeAgentOptions, HookMatcher

async def refund_guard(input_data, tool_use_id, context):
    amount = input_data["tool_input"].get("amount", 0)
    if amount > 500:
        return {"hookSpecificOutput": {
            "hookEventName": "PreToolUse",
            "permissionDecision": "deny",
            "permissionDecisionReason": "Refunds over $500 need a
human: call escalate_to_human with a handoff summary."}}
    return {}   # empty dict = allow unchanged

options = ClaudeAgentOptions(hooks={"PreToolUse":
[HookMatcher(matcher="mcp__billing__process_refund", hooks=
[refund_guard])]})
```

- **Callback signature:** `(input_data, tool_use_id, context)`; `input_data` has `tool_name`, `tool_input`, `hook_event_name` (plus `agent_id` / `agent_type` inside subagents).
- **`PreToolUse` outputs** (in `hookSpecificOutput`): `permissionDecision` = `"allow"` | `"deny"` | `"ask"` | `"defer"`, `permissionDecisionReason` (sent to Claude), `updatedInput` (rewrite arguments). `"defer"` ends the turn with `stop_reason: "tool_deferred"` so you can resume later.
- **`PostToolUse` outputs:** `additionalContext` (append to the tool result) and `updatedToolOutput` (replace the output before Claude sees it).
- **Precedence:** across hooks and permission rules, **`deny` > `defer` > `ask` > `allow`**. One `deny` blocks the call.

- **Prerequisite gate pattern:** keep state in your app (for example, "verified customer ID returned by `get_customer`"); a `PreToolUse` hook on `process_refund` denies until that state exists, with a reason that tells Claude which step to run first.
- **Deny + redirect:** the `permissionDecisionReason` should name the alternative path (escalate, ask for ID), so the model recovers instead of retrying.

EXAM TRAP
"Add a stronger instruction in the system prompt (MUST, ALWAYS)" is the wrong answer whenever the scenario needs **guaranteed** compliance (money, identity, safety). Use a hook or code-level gate.

6. Session state: resume, fork and named sessions

Operation	SDK	Claude Code CLI	Effect
Continue	`continue_conversation=True`	`claude --continue` (`-c`)	Most recent session in the current directory; same session ID
Resume	`resume=session_id`	`claude --resume <id-or-name>` (`-r`), `/resume`	Specific session; same ID; full prior context
Fork	`resume=id, fork_session=True` (`forkSession`)	`--resume <id> --fork-session`, `/branch`	**New** session ID with a copy of the history; original unchanged
Name	—	`claude -n <name>` / `--name`, `/rename <name>`	Resume later with `claude --resume <name>`

- **Fork use case:** from one shared analysis baseline, explore two divergent approaches (two testing strategies, two refactors) and compare, while the original stays resumable.
- **Fork is transcript-only:** the filesystem is shared. Forks that edit files collide unless isolated (worktrees, checkpoints).

DECISION RULE — RESUME VS START FRESH
Resume when prior context is **still valid**. If files changed since the analysis, **tell the resumed session exactly which files changed** for targeted re-analysis. If many prior tool results are **stale**, start a **new session with a structured summary** of the still-valid findings: stale tool output in history misleads the model more than a clean summary.

7. Structured tool errors (MCP `isError`)

Mechanics (grounded): an MCP tool result can set `isError: true`; SDK in-process tools return `{"content": [...], "is_error": True}` (Python) or `isError: true` (TS). A handler error does **not** stop the agent loop: an uncaught exception becomes an error result carrying the raw message, while a caught error lets you compose a message Claude can act on.

Design convention (application level, not MCP spec fields): put a structured payload in the error content so the agent can decide what to do next.

```
{ "isError": true,
  "content": [{ "type": "text", "text": "{\"errorCategory\":
\"transient\", \"isRetryable\": true, \"message\": \"Order service
timed out after 10s\", \"attempted\": \"lookup_order(id=8812)\",
\"suggestion\": \"Retry once; if it fails again, tell the customer
and escalate\"}" }] }
```

`errorCategory`	Example	`isRetryable`	Agent should
transient	Timeout, 503, rate limit	`true`	Retry with backoff (ideally locally inside the tool or subagent)
validation	Malformed date, missing field	`false` (until input fixed)	Fix the input and call again, or ask the user
permission	Caller lacks access	`false`	Stop that path; escalate or use an authorized route
business	Refund exceeds policy, account frozen	`false`	Explain the policy to the user and offer the allowed alternative

EXAM TRAP — ACCESS FAILURE VS VALID EMPTY RESULT
"No orders found" for a valid query is a **successful** result (`isError` false, `results: []`). A timeout is an **error**. Returning `[]` on a timeout makes the agent tell the customer "you have no orders": silent suppression. Returning a generic `"Operation failed"` hides whether a retry could work.

8. Tool distribution, scoping and `tool_choice`

- **Least tools per agent:** give each subagent only the tools for its role (`AgentDefinition.tools`). More tools = more schema tokens and more mis-selection; tools from another role invite misuse (a synthesis agent with web search starts re-searching instead of synthesizing).
- **Scoped variants:** instead of handing a broad tool to every agent, expose a narrow one where needed (for example, a read-only `verify_claim` for the synthesis agent while full search stays with the search agent). Route other cross-role needs back through the coordinator.

- **Replace generic tools with constrained ones** when misuse is the risk (a `fetch_order(order_id)` tool instead of raw SQL access).

`tool_choice` **(Messages API):**

Value	Behavior	Use for
`{"type": "auto"}`	Claude decides whether to call a tool (default when `tools` are provided)	Normal agent turns
`{"type": "any"}`	Must call **some** tool, Claude picks which	Guarantee a structured tool call when several output schemas exist
`{"type": "tool", "name": "extract_metadata"}`	Must call **that** tool	Force a mandatory first step (extract before enrich), then switch back to `auto`
`{"type": "none"}`	No tool calls (default when no `tools`)	Pure text turns

> **EXAM TRAPS**
> Forcing a tool every turn turns an agent into a fixed pipeline (it can never finish with `end_turn` text). Forced calls skip thinking. Manual extended thinking does not support `any` / `tool`, and some newer models reject them with a 400: use `auto` with strict tool definitions there (verify per model).

9. Escalation and ambiguity resolution

Escalate to a human when...	Do not use as the trigger
The customer **explicitly asks** for a human → honor immediately, without investigating first	**Sentiment** (frustration ≠ complexity; calm customers can have policy-gap cases)
Policy is silent or ambiguous for this request (exception, gap)	**Self-reported confidence** scores from the model (poorly calibrated)
The agent **cannot make meaningful progress** (repeated failures, missing authority)	"The case looks complex" without a concrete criterion

- **Write explicit criteria** in the system prompt, with **few-shot examples** of escalate vs resolve.
- **Frustrated but simple:** acknowledge the frustration and offer the fix; escalate if the customer repeats the request for a human.
- **Multi-concern requests:** split the message into distinct issues (e.g. a refund *and* an address change), investigate each one (in parallel where independent) against the **same verified customer context**, then reply with **one unified resolution** that addresses every item, rather than answering only the first concern.
- **Ambiguous identity:** if a lookup returns multiple matches, **ask for another identifier**; never pick by heuristic (most recent, first row).
- **Handoff summary:** the human usually cannot see the transcript, so pass a structured summary: customer ID, issue, root cause, what was tried, amounts, recommended action.

10. Error propagation in multi-agent systems

Subagent rule: recover locally from transient failures (bounded retries); propagate only what you cannot resolve, as **structured error context**:

```
{ "status": "partial", "failureType": "timeout", "attempted":
"search('EU AI Act fines 2025', source=news)",
  "partialResults": [ { "claim": "...", "source": "..." } ],
"alternatives": ["retry later", "use regulator site"] }
```

Anti-pattern	Why it fails	Fix
Silent suppression (return empty/success on failure)	Coordinator treats missing data as "nothing exists"	Mark the failure explicitly; keep partial results
Kill the whole workflow on one subagent failure	Throws away good partial work	Continue with what succeeded; annotate the gap
Generic status (`"search unavailable"`)	Coordinator can't choose retry vs alternative vs proceed	Failure type + attempted query + partial results + alternatives
Infinite retry in the subagent	Burns budget, hides an outage	Bounded retries, then propagate

- **Coverage annotations:** the final synthesis states which findings are well supported and which areas have gaps because sources were unavailable.

11. Long sessions and large-codebase context

Symptoms of context degradation: inconsistent answers, references to "typical patterns" instead of the specific classes found earlier, forgotten constraints.

Technique	How
Scratchpad file	Agent writes key findings (file paths, decisions, facts) to a file and re-reads it for later questions
Subagent delegation	Verbose exploration ("find all test files", "trace refund flow") runs in a subagent; only the summary returns
Phase summaries	Summarize phase N's findings, then inject that summary into the prompts of phase N+1's subagents
Checkpoint manifest	Workers persist structured checkpoints on disk; after an interruption, the orchestrator reloads that manifest to resume pending steps
`/compact`	Compress a long interactive session when verbose discovery output fills context (`/clear` when switching tasks)
Case-facts block	Keep exact amounts, dates, IDs and customer-stated expectations in a persistent block outside summarized history (summaries blur numbers)
Trim tool output	Keep only the relevant fields (5 of 40) before results accumulate
Position	Put key findings first and use clear section headers (models miss details buried in the middle of long inputs)

12. Provenance and multi-source synthesis

- **Claim→source mappings:** subagents return each claim with its source URL or document name, the relevant excerpt, location (page/section) and **publication or collection date**. The synthesis agent must **preserve and merge** these mappings; summarizing without them loses attribution.

- **Conflicts:** when credible sources disagree, report **both values with their sources** and annotate the conflict; don't silently pick one. Let the coordinator decide how to reconcile.
- **Dates prevent false contradictions:** 2021 vs 2024 figures may both be right; require dates in structured outputs.
- **Report structure:** separate **well-established** from **contested** findings; keep the original methodology context.
- **Match format to content:** use tables for quantitative metrics, narrative paragraphs for event summaries, and bulleted lists for engineering notes rather than forcing a single layout.

```
{ "claim": "Segment revenue grew 12% YoY", "source":
"https://example.com/10-K-2024", "excerpt": "…increased 12%…",
  "location": "p. 41", "published": "2025-02-14",
"conflicts_with": [{ "source": "analyst note", "value": "9%",
"published": "2024-11-02" }] }
```

13. Managed Agents

All Managed Agents endpoints require the beta header `managed-agents-2026-04-01` (set automatically by `client.beta.agents` / `environments` / `sessions`).

Primitive	Endpoint & mechanics
1. Agent	`POST /v1/agents` — versioned definition (`model` , `system` string — no `CLAUDE.md` hierarchy, `tools` , `mcp_servers` , `skills`). Built-in sandbox tools use **`{"type": "agent_toolset_20260401"}`** (in `/workspace`); custom tools (`{"type": "custom", ...}`) pause the session and emit **`agent.custom_tool_use`** until your backend posts **`user.custom_tool_result`** . Per-tool `permission_policy` (`always_allow` , `always_ask` , `auto`) gates execution via **`user.tool_confirmation`** .
2. Environment	`POST /v1/environments` — Anthropic-managed cloud container (`type: "cloud"` , `networking: {"type": "limited"}` allowlists) or self-hosted runner.
3. Session	`POST /v1/sessions` — binds a pinned `agent` version + `environment_id` , mounted `resources` (`file` , `github_repository` , `memory_store`), **`vault_ids`** (Vaults inject MCP/API credentials into outbound requests outside the container so raw secrets never enter the sandbox), and a spend `budget` .
4. Events	`POST /v1/sessions/{id}/events` & SSE `/events/stream` — `user.message` , `agent.message` , `agent.custom_tool_use` → `user.custom_tool_result` , `user.tool_confirmation` , `session.status_idle` with `stop_reason.type == "end_turn"` .

> **COMPLIANCE TRAP**
> **Managed Agents is stateful by design** (conversation history, container state and outputs stored server-side) and **cannot be used under a HIPAA BAA or Zero Data Retention (ZDR) agreement**. For regulated PHI or ZDR-governed data, run the loop yourself (Agent SDK or Messages API) on an eligible configuration.

14. Failure modes and exam traps

Symptom / scenario	Root cause → correct fix	Tempting wrong answer
Loop exits early whenever Claude says "Let me check..."	Termination keyed on text → loop on `stop_reason == "tool_use"` , stop on `end_turn`	Raising the iteration cap
Coordinator never delegates	`Agent` (alias `Task`) tool not in context/approved → add `"Agent"` to the explicit `tools` list (and `allowed_tools`)	Rewriting subagent descriptions only
Synthesis subagent asks "which sources?"	Subagents don't inherit parent context → pass full findings + metadata in the Agent prompt	Giving the subagent the parent's transcript file name
3 research subagents run one after another	Calls spread over turns → emit multiple `Agent` calls in **one** response	Switching to a bigger model
Agent refunds $900 despite "never above $500" in the prompt	Prompt compliance is probabilistic → `PreToolUse` hook denies and redirects to escalation	Repeating the rule in capitals
Customer told "you have no orders" during an outage	Tool returned `[]` on timeout → `isError` with `transient` , `isRetryable: true`	Retrying forever silently
Report presents one revenue figure; sources disagreed	Synthesis dropped conflict → keep both values with sources and dates, annotate	Averaging the two numbers
Resumed session reasons about functions that were deleted	Stale tool results → tell it which files changed, or start fresh with a summary	`/compact` the stale session
Escalations driven by an "angry" score miss policy-gap cases	Sentiment ≠ complexity → explicit criteria + few-shot examples	Lowering the sentiment threshold
Multi-tenant SDK app leaks memory across tenants or fails with `ENOENT` in TS	`setting_sources=[]` still loads auto memory, `~/.claude.json` , claude.ai MCP; TS `env` replaces `process.env`	`mcp_servers={}` without `strict_mcp_config=True`
PHI workload on Managed Agents "with deletion after each run"	Managed Agents is not BAA/ZDR eligible → Agent SDK / Messages API on eligible setup	Picking a US region

15. Glossary

Term	Definition
`stop_reason`	Why a response ended: `tool_use` (run tools, continue), `end_turn` (done), `max_tokens` , `pause_turn` , `refusal` , `stop_sequence` , `model_context_window_exceeded`
`ResultMessage.subtype`	SDK loop outcome: `success` , `error_max_turns` , `error_max_budget_usd` , `error_during_execution` , `error_max_structured_output_retries`
Hub-and-spoke	Coordinator routes all communication between isolated subagents
`Agent` tool (alias `Task`)	Tool Claude uses to spawn a subagent; must be available/approved for the coordinator
`AgentDefinition`	Programmatic subagent: `description` , `prompt` , `tools` , `model` , plus optional limits
`HookMatcher`	Registers SDK hook callbacks for an event and tool-name matcher
`permissionDecision`	`PreToolUse` hook verdict: `deny` > `defer` > `ask` > `allow`
`fork_session`	Copies a session's history into a new session ID; original unchanged; filesystem not forked

Term	Definition
`isError`	MCP tool-result flag marking a failed call (`is_error` in Python SDK tools)
`errorCategory` / `isRetryable`	App-level error payload fields: transient, validation, permission, business
`tool_choice`	`auto` / `any` / `tool` (forced) / `none`

Term	Definition
Claim→source mapping	Claim + source + excerpt + location + date, preserved through synthesis
Manifest	Known-location state export each agent writes so a coordinator can resume after a crash
`agent_toolset_20260401` **& Vaults**	Managed Agents built-in toolset (`/workspace`) and server-side credential injection (`vault_ids`)